

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Pseudocode Writing Task

Course Code:	ITA05	Course Title:	Computer Vision
CO Assessed:	CO5 – Naturalization (independent application using OpenCV)P5	Assessment:	Assessment Tool 2 – Pseudocode Writing Task
Weightage:	25% of CIA	Total Marks:	100
CO4 Statement:	Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL6)		
Student Name:	Galla Sai Siddartha	Reg. No.:	192425093
Section / Batch:	2024 - AIML	Date:	24 - 08 - 2026

Code of Conduct

I Galla Sai Siddartha (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Galla Sai Siddartha

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation	
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach	
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs	
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear	
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient	

Question 1: Real-Time Face Detection and Recognition Pipeline

1. Problem Understanding

The system processes a continuous live video stream from a camera to perform real-time face detection and recognition. The inputs to the pipeline are the live camera feed and a pre-trained dataset of known face images stored in a labelled directory. The outputs are annotated video frames displaying bounding boxes around every detected face along with the recognised identity label and confidence score, or the label 'Unknown' when no match is found. The pipeline must satisfy three core constraints: it must operate continuously without manual intervention, it must minimise processing delay to achieve near real-time frame rates, and it must handle multiple faces appearing simultaneously within a single frame.

2. Algorithm Design

The pipeline is structured as six independent modules orchestrated by a main loop. The first module, `LoadKnownFaces`, reads each labelled image from the known faces directory, converts it from BGR to RGB colour space, detects the primary face location, computes its 128-dimensional facial encoding using a deep convolutional network, and stores the encoding alongside its filename label. This module runs once at startup, eliminating repeated disk I/O during the live processing loop.

The second module, `PreprocessFrame`, resizes each captured frame to 50 percent of its original dimensions before any detection is applied. This downscaling halves the pixel area on which face detection operates, directly reducing the computational load of the HOG-based detector by a factor of four while preserving sufficient resolution for accurate face localisation. The frame is simultaneously converted from BGR to RGB, since the face recognition library operates on RGB input.

The third module, `DetectFaces`, applies the Histogram of Oriented Gradients face detector to the preprocessed frame and returns a list of bounding box coordinates for every face found. The fourth module, `RecognizeFaces`, computes a 128-dimensional encoding for each detected face region and measures its Euclidean distance against every encoding in the known faces database. A distance threshold of 0.5 is applied: faces with a minimum distance at or below this threshold are assigned the corresponding known label and a confidence score computed as $(1 - \text{distance}) \times 100$; faces exceeding the threshold are labelled Unknown.

The fifth module, `DrawResults`, scales all bounding box coordinates back to the original frame dimensions using the inverse of the downscaling factor, draws a coloured rectangle around each face (green for recognised, red for unknown), and overlays the identity label with confidence on a filled background rectangle for legibility. The sixth module, `DisplayFPS`, computes and renders the instantaneous frames-per-second count using the elapsed time between consecutive loop iterations.

3. Use of Control Structures

The main pipeline employs an infinite LOOP CONTINUOUSLY structure that terminates only when the user presses the 'q' key or the Escape key, satisfying the continuous operation constraint. A frame-skipping toggle variable `process_this_frame` alternates between TRUE and FALSE on every iteration, so the computationally expensive detection and recognition modules execute only on every other frame. This halves their processing burden and doubles the effective display frame rate, directly addressing the minimise processing delay constraint.

Within the LoadKnownFaces module, a FOR EACH loop iterates over every image file in the known faces directory, with an IF NOT EMPTY guard preventing encoding computation on images that contain no detectable face. Within RecognizeFaces, a FOR EACH loop paired with a ZIP iterator processes each detected face encoding in parallel with its bounding box location, and an IF-ELSE branch applies the distance threshold decision. Within DrawResults, a FOR EACH loop iterates over all recognition results, handling multiple simultaneous faces with a single IF-ELSE branch that selects the appropriate box colour and label text.

4. Pseudocode Structure and Modular Design

The pseudocode is organised into six named MODULE blocks followed by a MAIN PIPELINE block. Each module declares its INPUT and OUTPUT contracts explicitly, making inter-module dependencies clear and enabling independent testing of each component. The LoadKnownFaces module is fully decoupled from the real-time loop, the PreprocessFrame and DetectFaces modules are stateless frame-level transformations, and the RecognizeFaces module is the only module that depends on the pre-loaded known faces database. This separation ensures that any individual module can be replaced or upgraded without modifying the main pipeline structure. Indentation is consistent throughout, BEGIN and END delimiters mark every block boundary, and all data flow is expressed through named variables with directional arrows.

5. Completeness and Edge Case Handling

The pipeline handles all specified cases and several edge conditions. At startup, an explicit check on whether `known_encodings` is empty causes the system to print a descriptive error and exit cleanly rather than crash during the recognition step. Camera initialisation failure is detected immediately after OPEN_CAMERA and reported before the main loop begins. Within the loop, a frame capture failure check with a CONTINUE statement allows the pipeline to recover from momentary camera errors without exiting. The multiple-faces constraint is satisfied by iterating over all elements of the `face_locations` and `face_encodings` lists rather than processing only the first detected face. The Unknown label path ensures graceful handling of faces not present in the training dataset. Camera release and window destruction are performed after the loop exits to prevent resource leaks.

Detailed Pseudocode:

```

MODULE: LoadKnownFaces
INPUT : known_faces_dir (path to folder of labelled images)
OUTPUT : known_encodings (list), known_labels (list)
BEGIN
  known_encodings <- []
  known_labels <- []
  FOR each image_file IN known_faces_dir DO
    image <- READ_IMAGE(image_file)
    rgb_image <- CONVERT_BGR_TO_RGB(image)
    face_locs <- DETECT_FACE_LOCATIONS(rgb_image)
    IF face_locs IS NOT EMPTY THEN
      encoding <- COMPUTE_FACE_ENCODING(rgb_image, face_locs[0])
      label <- EXTRACT_FILENAME_WITHOUT_EXTENSION(image_file)
      APPEND encoding TO known_encodings
      APPEND label TO known_labels
    END IF
  END FOR
  RETURN known_encodings, known_labels
END

```

```

MODULE: PreprocessFrame
INPUT : frame (raw BGR frame from camera)
OUTPUT : rgb_small (resized RGB frame), scale_factor
BEGIN
  scale_factor <- 0.5
  small_frame <- RESIZE(frame, scale_x=scale_factor, scale_y=scale_factor)
  rgb_small <- CONVERT_BGR_TO_RGB(small_frame)
  RETURN rgb_small, scale_factor
END

```

```

MODULE: DetectFaces
INPUT : rgb_small (preprocessed frame)
OUTPUT : face_locations (list of bounding boxes)
BEGIN
  face_locations <- DETECT_FACE_LOCATIONS(rgb_small, model="hog")
  RETURN face_locations
END

```

```

MODULE: RecognizeFaces
INPUT : rgb_small, face_locations, known_encodings, known_labels
OUTPUT : results (list of (label, confidence, location) tuples)
BEGIN
  results <- []
  face_encodings <- COMPUTE_FACE_ENCODINGS(rgb_small, face_locations)
  FOR EACH face_encoding, face_loc IN ZIP(face_encodings, face_locations) DO
    distances <- COMPUTE_FACE_DISTANCES(known_encodings, face_encoding)
    min_dist <- MIN(distances)
    best_index <- ARGMIN(distances)
    IF min_dist <= THRESHOLD (0.5) THEN
      label <- known_labels[best_index]
      confidence <- (1 - min_dist) * 100
    ELSE
      label <- "Unknown"
      confidence <- 0.0
    END IF
    APPEND (label, confidence, face_loc) TO results
  END FOR
  RETURN results
END

```

```

MODULE: DrawResults

```

```

INPUT : frame, results, scale_factor
OUTPUT : annotated_frame
BEGIN
  FOR EACH (label, confidence, face_loc) IN results DO
    top, right, bottom, left <- face_loc
    top <- top / scale_factor
    right <- right / scale_factor
    bottom <- bottom / scale_factor
    left <- left / scale_factor
    DRAW_RECTANGLE(frame, (left, top), (right, bottom), color=GREEN, thickness=2)
    IF label != "Unknown" THEN
      display_text <- label + " (" + ROUND(confidence, 1) + "%)"
      box_color <- GREEN
    ELSE
      display_text <- "Unknown"
      box_color <- RED
    END IF
    DRAW_FILLED_RECTANGLE(frame, (left, top-25), (right, top), color=box_color)
    DRAW_TEXT(frame, display_text, position=(left+4, top-6),
      font=FONT_HERSHEY_SIMPLEX, scale=0.55, color=WHITE, thickness=1)
  END FOR
  RETURN frame
END

```

MODULE: DisplayFPS

```

INPUT : frame, fps_value
OUTPUT : frame with FPS overlay
BEGIN
  fps_text <- "FPS: " + ROUND(fps_value, 1)
  DRAW_TEXT(frame, fps_text, position=(10, 25),
    font=FONT_HERSHEY_SIMPLEX, scale=0.6, color=YELLOW, thickness=2)
  RETURN frame
END

```

MAIN PIPELINE: RealTimeFaceDetectionAndRecognition

```

BEGIN
  known_encodings, known_labels <- CALL LoadKnownFaces("known_faces/")
  IF known_encodings IS EMPTY THEN
    PRINT "ERROR: No known faces loaded. Exiting."
    EXIT
  END IF
  camera <- OPEN_CAMERA(device_id=0)
  SET camera PROPERTY FRAME_WIDTH = 640
  SET camera PROPERTY FRAME_HEIGHT = 480
  IF camera IS NOT OPEN THEN
    PRINT "ERROR: Cannot open camera. Exiting."
    EXIT
  END IF
  prev_time <- GET_CURRENT_TIME()
  process_this_frame <- TRUE
  LOOP CONTINUOUSLY DO
    success, frame <- READ_FRAME(camera)
    IF NOT success THEN
      PRINT "WARNING: Frame capture failed. Retrying."
      CONTINUE
    END IF
    IF process_this_frame = TRUE THEN
      rgb_small, scale_factor <- CALL PreprocessFrame(frame)
      face_locations <- CALL DetectFaces(rgb_small)
      results <- CALL RecognizeFaces(rgb_small, face_locations,
        known_encodings, known_labels)
    END IF
  END LOOP

```

```
process_this_frame <- NOT process_this_frame
annotated_frame  <- CALL DrawResults(frame, results, scale_factor)
curr_time       <- GET_CURRENT_TIME()
fps             <- 1.0 / (curr_time - prev_time)
prev_time       <- curr_time
annotated_frame  <- CALL DisplayFPS(annotated_frame, fps)
SHOW_WINDOW("Real-Time Face Detection and Recognition", annotated_frame)
key <- WAIT_KEY(1ms)
IF key = 'q' OR key = ESC THEN
  BREAK
END IF
END LOOP
RELEASE_CAMERA(camera)
DESTROY_ALL_WINDOWS()
END
```